

A Critical Assessment of Testability in Communication Protocols

Antonio A.F. Loureiro^{2,3}

Samuel T. Chanson^{1,3}

*Son T. Vuong*³

⁽¹⁾Hong-Kong University of Science and Technology
Department of Computer Science
Clear Water Bay, Kowloon, Hong-Kong

⁽²⁾Universidade Federal de Minas Gerais
Departamento de Ciência da Computação
Belo Horizonte, MG, Brazil

⁽³⁾University of British Columbia
Department of Computer Science
Vancouver, BC, Canada

Resumo

O conceito de projeto visando testabilidade tem sido aplicado por projetistas de hardware desde a década de 70. Este princípio também tem sido aplicado no desenvolvimento de software. Na área de hardware este princípio já alcançou um certo grau de maturidade com técnicas bem definidas, padrões, ferramentas, etc. No entanto, na área de software, este é um assunto relativamente novo e somente recentemente apareceram algumas propostas para incorporar testabilidade ao software. Dentre as diversas áreas em software que têm estudado projeto visando testabilidade, a área de protocolos de comunicação tem se destacado.

Neste trabalho nós apresentamos uma análise crítica dos diferentes métodos de testabilidade propostos para protocolos de comunicação e apresentamos sugestões que podem ajudar no desenvolvimento futuro de novos métodos nesta área. Para alcançar este objetivo, várias perguntas relacionadas com este tema são respondidas.

Abstract

The concept of design for testability (DFT) was incorporated by hardware designers since the 1970's. This idea was also taken to software. But contrary to hardware where this subject has already reached a certain degree of maturity with well-known procedures, standards, tools, etc., DFT in software is a relatively new research area and only recently some proposals have been made. Among the different problem domains in software that have studied DFT, communication protocols has been an active research area.

In this paper we make a critical assessment of the different testability methods proposed to communication protocols so far and present suggestions that can help to guide future developments in this area. In order to achieve that we answer several related questions to this subject.

1 Introduction

Hardware designers have realized early that some design principles are needed to overcome the ever increasing complexity of testing integrated circuits (ICs). ICs should be designed to contain facilities to make the circuit more easily testable and consequently manufacturable. Presently, testability methods are applied before or concurrently with the system logic design and not as an afterthought when the circuit is completed. This is called design for testability (DFT). These methods propose circuit modifications or addition of special test circuitry on the IC to allow better test coverage.

The concept of design for testability was also taken to software. But contrary to hardware where this subject has already reached a certain degree of maturity with well-known procedures, standards, tools, etc., DFT in software is a relatively new research area and only recently some proposals have been made. Among the different problem domains in software that have studied DFT, communication protocols has been an active research area.

Communication protocols are concurrent reactive systems and it is recognized that testing of concurrent programs is harder than testing sequential ones as explained in the next section. Therefore it is not a surprise to see different groups studying the problem of design for testability in communication protocols.

The goal of this paper is to make a critical assessment of the different methods proposed so far and present suggestions that can help to guide future developments in this area. In order to achieve that we answer the following questions:

1. Why is the testing of concurrent reactive systems so difficult?
2. How are communication protocols tested?
3. What does design for testability mean?
4. What is the current state of the art of DFT in communication protocols?
5. Is DFT = observability and controllability?
6. What are the directions to follow?

Basically, this paper is divided into three parts. The first one is an introductory part that describes the problem of testing concurrent reactive systems (Section 2) and communication protocols (Section 3), and discusses the meaning of DFT (Section 4). This part will help us to understand the others. The second one makes a critical assessment of DFT proposals in communication protocols (Sections 5 and 6). Finally, the third one presents some directions to tackle this problem (Section 7). In Section 8 we present our concluding remarks.

2 Why is the testing of concurrent reactive systems so difficult?

In this section we look at the problem of testing concurrent reactive systems and present five reasons that make the testing of this kind of software more difficult to be realized.

Software systems can be broadly partitioned into two categories: transformational and

reactive. A *transformational* software is the usual type of program which often produces a result at the end of a computation. They are transformational in the sense that they transform an initial state to a final state. Examples of transformational programs are sorting routines, off-line data processing, and numerical packages.

A reactive program does not produce a final result but maintains a continuous interaction with its environment. Examples of reactive programs are operating systems, communication protocols, and programs controlling mechanical or chemical processes.

The problem of testing concurrent programs is in general harder than testing sequential ones. Some of the difficulties in testing concurrent programs are (i) the probe effect, (ii) non-reproducible behavior of the system, (iii) increased complexity, and (iv) lack of a synchronized global clock.

When the concurrent program is also reactive, another level of difficulty is added: (v) the build-up of states due to its continuous interaction with its environment. All these five reasons are discussed in more detail in the following.

The probe effect. A probe is a name given to a piece of hardware and/or software introduced into a system with the main purpose of observing its internal behavior with different purposes. For example, to gain better knowledge about its behavior or to make sure that some constraints are not violated.

The probe effect is a name given to the fact that the behavior of a system may be changed when attempting to observe it. This is the main impediment to achieve observability of a system.

The kind of effect caused by the introduction of a probe into a system depends on how its monitoring is performed: intrusive or interfering. Basically, the probe may interfere with the relative timing among processes, preventing certain timing or coordination related errors from occurring, or may introduce faults that would not be present without it.

There are at least three different ways of addressing the probe effect. It can be ignored, i.e., it is assumed that the effect will seldom or never occur; minimized if the probe is implemented efficiently; or avoided by implementing the probe through logical clock (i.e., hiding it), or leaving the probe in the production system (it may be difficult to avoid), or using dedicated hardware for the system.

Non-reproducible behavior of the system. It is very important to be able to reproduce the behavior of a system for a given input when testing or debugging it. Otherwise it will be more difficult to realize these tasks. This is the main assumption when performing regression testing in sequential software. In fact achieving reproducibility in sequential deterministic programs is relatively trivial since control path choices depend on inputs to the program and its initial state.

In concurrent software this is not the case. Its behavior is not reproducible because of nondeterminism either due to concurrency, or introduced in the specification, or due to non-observability. In fact, without any mechanism to control a nondeterministic selection made it is not possible to establish an upper bound on the number of attempts required to reproduce a particular computation.

The usual solution to testing of concurrent programs is to create a log file of event occurrences and replay it when debugging the system. This approach is not trivial, introduces the probe effect and may be difficult to achieve due to a lack of controllability.

Increased complexity. Complexity is increased because of a greater difficulty in understanding, specifying, and testing the ways processes interact among themselves in a distributed application. In other words, it is very difficult to anticipate all possible scenarios that might occur when nondeterminism, communication, and synchronization are taken into account.

Lack of a synchronized global clock. A distributed system can be viewed as a set of cooperating processes communicating with each other through message passing. In this situation it is generally not possible to determine precisely, i.e., deterministically the total order of events in the system because of the lack of a physically global clock. It is only possible to give a partial order. For example, a send event p must happen before the corresponding receive event in process q , and for some events there is no way to tell which one occurred first. This often leads to errors in specifications.

Build-up of states. As stated above, a reactive system interacts continuously with its environment. This problem is illustrated in the following. Consider a protocol implementation I where its local state can be defined informally as the values associated to its local variables and the contents of its communication channel at a given moment in time. The initial local State 0 is shown in Figure 1 with two circles. When a new event happens its local state changes. This is shown in Figure 1 through a transition labeled with $\langle e \rangle$. Usually, an event $\langle e \rangle$ is classified into one of the following three types: a send event, a receive event, or an internal event. A send event causes a message to be sent, and a receive event causes a message to be received. An internal event may be a timeout, an interrupt, etc. We are considering that the environment where I is embedded, i.e., operating system, etc., does not change the value of variables nor the contents of the communication channels.

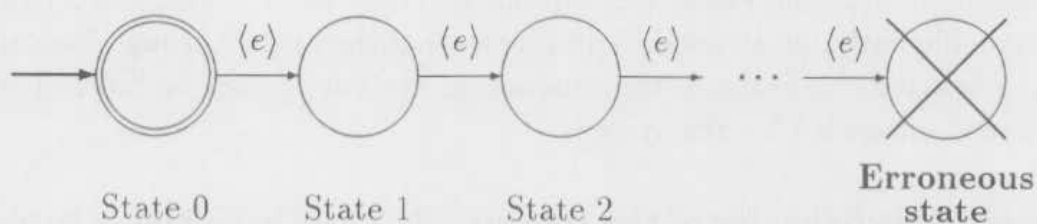


Figure 1: The problem of state build-up in concurrent reactive systems.

This example illustrates how valid input events $\langle e \rangle$ may lead to an erroneous state in the protocol implementation I . When the implementation I reaches an erroneous state it may either continue to run but producing erroneous output or may crash and stop.

An erroneous state was reached because the values associated to its local variables and the contents of its communication channel are not correct according to constraints or requirements given in the protocol specification. Note that this behavior can only be identified if test sequences are as long as the length of the faulty path.

In fact, Parnas, van Schouwen and Kwan [21] identify this situation as a critical problem to reactive safety-critical software. They suggest that this kind of software should be initialized periodically to avoid the build-up of states.

3 How are communication protocols tested?

The ideal way to prove (or disprove) the logical correctness of a design specification of a communication protocol is to apply some *verification* methods to check whether some desirable properties hold (e.g., safety and liveness properties). This is a complex problem and it is not difficult to show that the verification of a simple property such as absence of deadlock (safety property) is PSPACE hard [25]. When verifying the correctness of communication protocols we have to be aware of these complexity bounds. In practice, only some parts of a protocol are verified.

This means that we cannot avoid the testing process where a protocol implementation must be checked for conformity to its specification. Testing is also a difficult problem since exhaustive testing of implementations of complex protocols (e.g., OSI protocols) is both theoretically and practically not feasible. Therefore, the goal of conformance testing is to detect as much errors as possible in an implementation using the minimum number of test cases. However, most communication protocols have been designed and implemented without conformance testing in mind. Yet, it is estimated that, in general, “testing consumes at least half of the labor expended to produce a working program” [2].

In this section we present an overview of the activities involved in protocol testing and the test methods used.

Testing activities

The testing process involves some activities that are common to all types of protocol testing discussed above. The activities include:

- *Test generation*—the process of deriving test cases from a protocol specification.
- *Test verification*—the process of checking whether the behavior expected from a test case when applying to an implementation under test (IUT) is in fact valid with respect to the specification.
- *Test selection and parameterization*—A protocol implementation does not need to support all functions and features present in the specification. Therefore, only a subset of the possible test cases may be applied to the implementation. The process of identifying the correct subset of test cases is called test selection. Test parameterization is the process of assigning values to the parameters of the test cases.
- *Test execution*—The process of generating an executable test case from a selected and parameterized test case and applying it to the implementation. There should be a way to record the history of the test case execution for posterior analysis when it will be identified possible errors in the implementation.

Test methods

ISO IS9646 (OSI-CTMF) [17] has defined four abstract test methods for protocol conformance testing as shown in Figure 2: local (LTMs), distributed (DTMs), coordinated (CTMs), and remote (RTMs). All of them use a point of control and observation of ASPs below the IUT. Also, a lower tester separated from the system under test (SUT)

exchanges PDUs with the IUT. In the following, we give a brief overview of each abstract test method.

Local and distributed test methods. Both use a point of control and observation of the abstract service primitives (ASPs) at the upper service boundary of the IUT. In the LTMs, the upper tester is located within the test system whereas in the DTMs it is located within the IUT. The LTMs require the upper service boundary of the IUT to be a standardized hardware interface. The DTMs require it to be either a human user interface or a standardized programming language interface. In both methods, it is required to access this interface for testing purposes. In the LTMs, the test coordination procedures are realized entirely within the test system. In both methods, the requirements for the test coordination procedures are specified, but not the procedures themselves.

Coordinated and Remote test methods. Both test methods do not require access to the upper service boundary of the IUT. In the CTMs, the test coordination procedures are realized by test management protocols (TMPs). The upper tester is an implementation of the relevant TMP. In the RTMs, the abstract test suite (ATS) may imply or informally express requirements for test coordination procedures. Although, it is not possible to make any assumption regarding their feasibility or realization. There is no upper tester but some of its functions may be done by the SUT.

Figure 2 depicts test methods for single-layer IUTs. Variants of these methods can be applied to multi-layer IUTs, where an IUT is tested layer by layer. An embedded method is a special case of multi-layer testing where a lower layer protocol is wrapped around upper layer protocol data units (PDUs). In this case, we are only interested in testing the lower protocol that is accessible through upper layers.

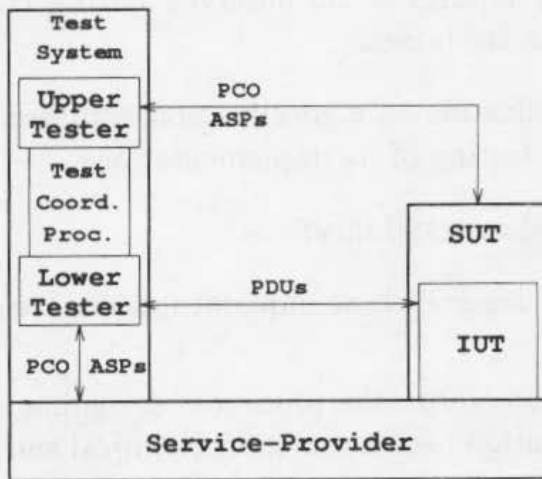
The Ferry Clip method is an approach to realize any of these abstract test methods [10].

4 What does design for testability mean?

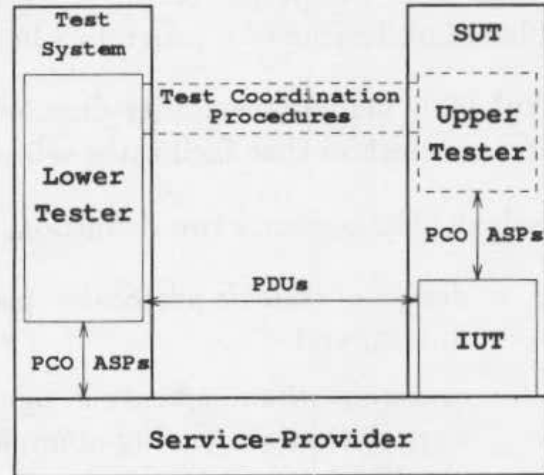
In this section we present several definitions to testability that have appeared in the literature and then we discuss the meaning of DFT. All work associated to communication protocols is indicated by the symbol *.

Definitions (in chronological order):

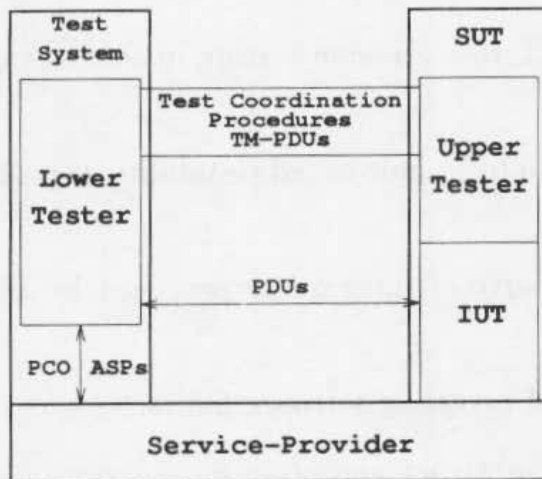
- Boehm et al. [5]: one of the eleven important characteristics that software should have. The other ten are understandability, completeness, conciseness, portability, consistency, maintainability, usability, reliability, structuredness, and efficiency.
- Boehm [6]: one of the four major software requirements and design verification and validation criteria. The other three are completeness, consistency, and feasibility.
- Hoffman [16]: does not give explicitly a definition for testability but identifies observability and controllability as two important factors in DFT.
- Bache and Müllerburg [1]: the effort needed for the testing process.
- Beizer [2]: in terms of goals: to reduce the work of testing, and to have a testable code with fewer errors.



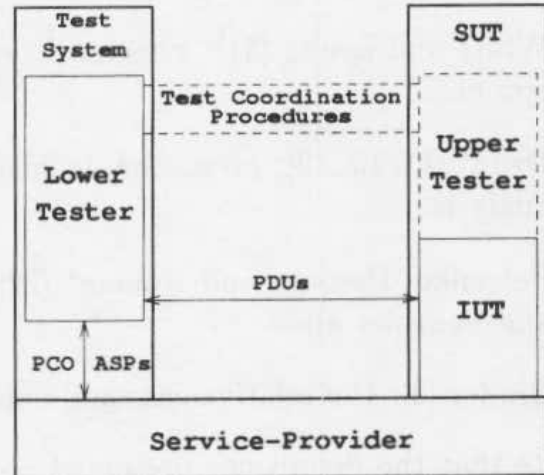
(a) Local test method.



(b) Distributed test method.



(c) Coordinated test method.



(d) Remote test method.

Figure 2: Test methods.

- Dssouli and Fournier* [14]: property of the software that “includes testing facilities allowing the easy application of several testing methods, the detection of existing errors and their more rapid correction.”
- Larsen [19]: one of the four criteria to compare specification formalisms. The other three are expressivity, compositionality, and decidability.
- Probert and Geldrez* [24]: do not give explicitly a definition for DFT.
- Thayer and Thayer [27, page 667]: a software quality metric.
- Freedman [15]: gives a definition for domain testability, i.e., a specification with no input and output inconsistencies.
- Schütz [30]: compares the testability of two types of distributed architectures for real-time systems.

- Voas [28]: prediction of the tendency for failures to be observed during random black-box testing of a program when faults are present.
- Yu* [32]: defines testability-directed specification as a specification expressed in a formal method that facilitates subsequent testing of its implementations.
- Saleh* [26]: presents two definitions related to testability:
 - *design of testable protocols*—protocol designs whose implementations are easier to test, and
 - *communication software design for testability*—the process of designing a software so that the testing of implementation becomes more economical and manageable.

Note that the first definition is related to the design (specification) phase and the second one to the implementation.

- White and Leung [31]: systematic approach to regression testing in software development.
- Drira* [11, 13, 12]: presents a definition for a technique called testability degradation analysis.
- Petrenko, Dssouli, and Koenig* [22]: property of software as proposed by Dssouli and Fournier above.
- Binder [3]: the relative ease and expense of revealing software faults.

Note that the definitions presented are quite abstract and often do not state specific testing goals or improvements to be achieved. This is reflected in the lack of real proposals. Among the work listed above only Probert and Geldrez*, Freedman, Voas, Yu*, Saleh*, White and Leung, Drira* and Petrenko, Dssouli, and Koenig* propose concrete methods to improve software testability. Communication protocols have five concrete proposals out of six. The others discuss testability in a generic way.

When discussing testability, there is a question that arises naturally: is there a general technique that can be applied during the design process so the implementation derived from the specification will be more testable? The answer is definitely no. There are so many variables and conditions that need to be considered during the testing process that no single technique will cover all of them. This fact is also present in the hardware domain where there are different testability techniques that can be applied during the design process depending on the fault model considered. In [9] it is discussed the problem of testing and testability in hardware and software.

Testability means that a piece of software has some properties, characteristics, or features that will facilitate the testing process. Design for testability (DFT) is the process of applying techniques and methods during the design phase in order to reduce the effort and cost in testing its implementations. How easily the testing process can be realized depends on many factors. For instance, it depends on the goals of testing or the testing strategy (e.g., length of test sequences, testing time, and coverage); how the implementation under test is viewed: black-box, grey-box or white-box, and the problem domain or application. To develop better DFT techniques it is very important to understand the

two fundamental criteria: (i) the testing process, i.e., goals, strategy, architecture, etc., and (ii) the kind of application we are dealing with.

DFT should not be confused with testability or testable design where the design process rather than the implementation (somehow) can be tested. Testability is a property that should be introduced in the design for people who will test the product (a software or a hardware element) and not, in general, for the end users. (Depending on the application and/or requirements, some testing facilities can/should be made available to the end user.)

The testability methods discussed above can be grouped into the following classes:

1. Methods that assess the ease with which inputs can be selected to achieve some structural testing criteria.
2. Methods that predict the probability that existing faults will be revealed during testing given a specific criterion for generating inputs.
3. Methods that propose a design specification that will lead to an implementation that will be easier to test according to some testing goals and the application domain.

The first class includes methods that work at the implementation level. This contrasts to methods in the third class that work at the design level. The second class is intermediate and includes methods that work at both the implementation and design levels. The first two classes define what can be called software testability analysis and the third one design for testability analysis.

5 What is the current state of the art in DFT of communication protocols?

There are at least six proposals to enhance testability in communication protocols. In this section we make a critical review of them. Initially, the testing goals are grouped in five categories. For each category, the factors that affect and can improve the testing process are identified and then it is explained how the testing goals can be accomplished.

The testing goals of each method can be grouped in the following categories:

1. Semantic coverage: Probert and Geldrez [24].
2. Observability and controllability: Dssouli and Fournier [14], and Saleh [26].
3. Properties of the formal method used: Yu [32].
4. Testing environment: Drira et al. [11, 13, 12].
5. Length of test cases: Petrenko, Dssouli, and Koenig [22].

Semantic coverage

Probert and Geldrez [24] propose a grey-box testing technique called semantic instrumentation in order to enhance the software design for testability.

Basically, the method proposes to identify at the specification and design level, equivalence classes of behavior (ECB). These classes are used to group different types of functional requirements present in the specification, decisions taken at the design level, decisions to be taken at the implementation level, etc. Each ECB receives a unique label. This could be seen as a kind of module decomposition.

In order to implement this method, Probert and Geldrez propose to represent the design as a *design machine* (DM) that is an extension of finite state machines. A DM consists of states and transitions. States are design decision points (DDPs) and can be of three types: state DDP—describes the actual state of the application, input/output DDP—describes primitives exchanged at the interface, and error/recovery DDP—deals with the error/recovery aspects of the machine. Transitions are actions and can be of two types: yes/no—these are unconditional transitions that depend on the yes or no response to the current DDP decision, and yes/no lb—these are labeled transitions that depend on additional predicates for firing a transition.

At the implementation level, a code segment is written for each equivalence class identified in the design. A one-to-many mapping function is established from equivalence classes to code segments. A special code called semantic probe is added to each code segment corresponding to a class to detect whenever the code is executed and record this fact in terms of the corresponding design ECB.

The mapping is done to guarantee that design decisions, functional requirements, etc. are tested accordingly. The method can be used for documenting and verifying the completeness and consistency of the application.

Observability and controllability

Both Dssouli and Fournier [14], and Saleh [26] look at testability considering observability and controllability. Furthermore, both make use of a testing interface to observe and control the implementation under test.

Dssouli and Fournier propose the introduction of internal interaction points (IPs) during the instrumentation and testability phase so the implementation structure of a communication software can become observable during the testing process. This phase is introduced before the implementation but after the design.

In their case, IPs do not need to be used for testing purposes only. Furthermore, the IPs can be either passive or active. A passive testing interface is just an interface of observation whereas an active testing interface is an interface of control and observation. They suggest three alternatives to improve O&C: addition of primitives, selective broadcasting, and direct call to a trace procedure without discussing them in details.

The first alternative (addition of primitives) seems to be the most appropriate to implement. The second alternative changes the definition of an IP as the FDTs Lotos, Estelle, and SDL understand. The third alternative is not flexible since the code must be re-compiled every time a new call is added which may produce side-effects. In this case the implementer has to make sure that the call does not interfere with the original module.

Saleh defines architectural and behavioral requirements in order to enhance the observability and controllability of a communication protocol. The architectural requirement is accomplished through a testing interface called testability service access (T-SAP). The behavioral requirement defines four primitives that can be used exclusively at the T-SAP.

The primitives related to observability are Probe (P) to probe the state of a protocol entity, and Trace (T) to enable the trace collection of a given entity. The primitives related to controllability are Restart or Reset (R) to reset a given protocol entity to its initial state, and Set (S) to set a protocol entity to a particular state according to the variables or parameters specified in the service primitive.

Saleh points out that testability requirements introduced into the original protocol design (procedure rules) make it more complex and consequently increase the complexity and cost of its validation. On the other side, the complexity of a protocol derived from a testable service using some synthesis technique should not be of concern since its correctness is guaranteed by applying some synthesis method, i.e., the protocol is correct by construction.

It should be noted that the complexity of a protocol design is increased when testability requirements introduced into the design enhance its functionality in order to achieve some testing goal. If the testability requirements impose constraints on the design then the complexity of validation and testing may in fact decrease.

Saleh classifies the service primitives into passive and active. Both the T-SAP and the service primitives can be used only for testing purposes.

Note that the simple existence of a testing interface and specific testing primitives do not guarantee the observability and controllability of the implementation under test. It is very important that the implementation design allows the observability and controllability of the IUT. Of course none of this can be done without the expertise of the tester. Therefore, the tester plays a central role in this approach.

Also note that observability and controllability are in general related to a testing interface and primitives. This idea is also present in design for testability in the hardware domain. In fact it is one of basic principles to be achieved when testing integrated circuits.

Both approaches introduce O&C using a synthesis technique but at different levels of abstraction. Dssouli and Fournier before the implementation and Saleh during the specification of the communication services.

The method proposed by Dssouli and Fournier can be applied to a formal or informal specification whereas the method proposed by Saleh supposes that the specification is written using a formal method.

Among the two methods, the one devised by Saleh is the most difficult to implement. The reason is that the current state of the art in synthesis techniques has not yet reached the point where a complete protocol specification can be derived automatically from its service definition [23]. The degree of difficulty to implement the ideas proposed by Dssouli and Fournier depends on the alternative chosen.

Regarding the probe effect both methods affect the testing process. Dssouli and Fournier, and Saleh perform what can be called an instrumentation of the implementation. This process is not completely passive and therefore can change the behavior of the testing process when compared to the same implementation that does not have any facilities to support observability and controllability. Only Saleh recognizes this point. This is because there is some overhead associated with the observability and controllability facilities (e.g., to test whether the service primitive is one used for observability and controllability). Even if these facilities are left in the *production* version of the protocol, they may affect the behavior of the protocol because the overhead is still present, despite the fact they will not be used during normal operation.

Properties of the formal method used

Yu [32] identifies two requirements that a formalism should satisfy in order to have a testability-directed specification:

1. Strong expressive power—capacity of expressing concurrent behaviors that can be observed during the testing process.
2. Timing model that facilitates the testing process—a model that facilitates the generation of test cases.

Yu proposes an extension to Lamport's work on logical clocks [18] called relative clock time as a basis for specifying and testing distributed systems. This extension together with temporal assertions are applied to Parnas' trace assertion language (TAL) [20] yielding an extended trace assertion language (ETAL). TAL is a language used to specify a software module or system in terms of its external observable behavior. ETAL allows to specify a system in terms of relative time.

The relative clock time is the basis for the definition of the global events model that in turn is the basis for a new specification-based test result analysis method. This method can be applied to conformance testing of both service and protocol specifications. In particular, the global events model can be used to detect collisions in communication protocols. Collisions can be viewed as *cross overs* of interactions in a distributed system. Collisions may or may not cause testing problems.

The specification-based test result analysis method enhances testability by identifying spontaneous events and reconciling locally inconclusive test results resulting from collisions.

Testing environment

Drira et al. [11, 12] looked at the following problem: let I be an implementation derived from a specification S expressed in labeled transition systems (LTS) that is embedded in an environment E . The testability of S is said to be good when the sets of rejected implementations obtained testing I through E and when testing I directly, i.e., without E , are exactly the same. They proposed a method for analyzing the testability of I when it can only be tested through E . It is shown that faulty implementations may be not detected when tested through E but could be if were tested directly. This is called testability degradation analysis.

In general, communication protocols are embedded in some kind of environment: operating system, a specific application, etc. Therefore, this kind of analysis is very important in critical and non-critical embedded systems.

Drira et al. introduced the concept of refusal graph as the basis for testability degradation analysis in labeled transition systems. This analysis can be applied to formal description techniques such as Lotos. The refusal graph is the mathematical tool to perform the testability degradation analysis. This graph represents a communicating system in an abstract way. There are two types of labels in a refusal graph: actions associated to transitions, and refusal sets associated to states. Figures 3.(a), 3.(b), and 3.(c) show the three main possibilities of nondeterminism that a test cannot distinguish in labeled transition systems. In Figure 3.(d) is given their common refusal graph. The symbol τ represents an unobservable transition. This refusal graph shows that the system must

first execute action a , then execute either action b or c , but it can be in an internal state refusing one of these actions.

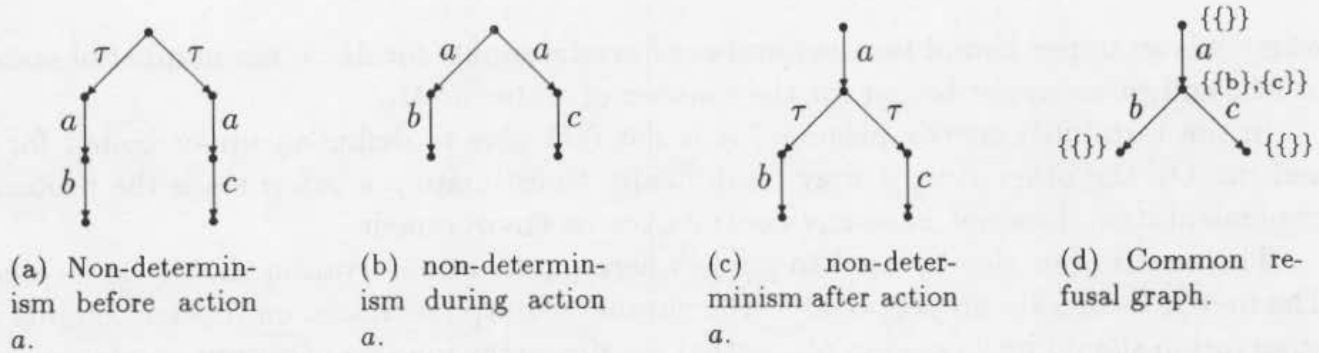


Figure 3: A common refusal graph to three labeled transition systems.

Length of test cases

Petrenko, Dssouli, and Koenig [22] proposed a testability metric to compare different protocol implementations derived from the same protocol specification. The implementations are built as a composition of finite state machines (FSMs). The metric only considers the control part of the protocol specification that is also modeled as an FSM. This may be a problem if the complexity of the protocol is “hidden” behind the data part.

One of the problems in protocol testing is the identification of the best test suite with minimal length and maximal coverage of faults. A test set may be infinite since a communication protocol is a reactive program and complete test coverage cannot be achieved unless a fault domain be defined a priori.

Petrenko, Dssouli, and Koenig use this observation to propose a testability metric for protocol implementations that is inversely proportional to the amount of testing effort. In this case the testing effort is considered to be the length of a test suite needed to achieve complete fault coverage assuming a pre-defined fault domain.

The length of this test suite is based on the concept of a complete test suite. A test suite is complete if can distinguish among conforming and faulty implementations of a protocol specification the faulty ones. Of course the testing process becomes infinite if it is not imposed an upper bound on the number of faults.

To become this process finite a fault domain based on the number of states of both specification and implementation is defined. Let M_s be an FSM that represents a protocol specification with n states. Let M_i be an FSM that represents a protocol implementation with k states, where $k \leq m$ and $m \geq n$. A complete test suite can distinguish all faulty implementations with k states.

The authors assume that M_s is reduced, completely specified, deterministic and strongly connected, and all implementations derived from M_s have a reliable reset function that can take the implementation to its initial state. These are strong assumptions about the characteristics of M_s . In general, most protocol specifications do not satisfy these assumptions [4].

Petrenko, Dssouli, and Koenig propose a conjecture on how to estimate the length of complete test suites for machines M_i that is the length provided by a complete test suite

divided by the number of outputs of M_i . The testability metric $\mathcal{T}$ can be derived directly:

$$\mathcal{T} = \frac{o}{mn^2p^{m-n+1}}$$

where p is an upper bound for the number of invalid inputs for M_i , n the number of states in M_s , and m an upper bound for the number of states in M_i .

In the testability metric proposed it is not difficult to define an upper bound for n and m . On the other side, it may be difficult to estimate p a priori since the protocol implementation does not have any control over its environment.

This metric can also be used to guide where points of observation should be located. The metric is directly proportional to the number of outputs. Based on this fact, points of observation should be located in places that maximize the number of output combinations.

Putting the methods into perspective

In the following we put all the six methods in perspective so it will be easier to have a global view of them.

Table 1 shows for each work the testing goals (**G**), the factors that affect (**A**) and improve (**I**) testability, and how (**H**) the testing goals can be achieved. If the work does not discuss a given item then it is not presented in the table.

Table 2 compares the six methods using other criteria. The column DFT/T shows whether each method applies testability methods during the design or not. The column Phases shows the phases affected by each method: S for specification, D for design, I for implementation, and T for testing. The column Tech. indicates whether the method uses synthesis (S) or analysis (A) techniques. The column Probe effect indicates whether the probe effect occurs when observability and controllability are introduced into the system or not. The column Metrics indicates whether some testability metric is defined or not. The column Tool indicates whether there is a tool available that implements the method or not.

6 Is DFT = observability and controllability?

There is a tendency to associate design for testability to observability and controllability. Among the definitions presented in Section 4, Hoffman, Dssouli and Fournier, Freedman, and Saleh look at testability considering observability and controllability. Schütz, Voas, Petrenko et al., and Binder also consider O&C in their methods.

Probably this fact comes originally from the hardware domain where DFT capabilities are developed to have adequate observability and controllability of the circuit under test. The main reason to have an observable and controllable circuit is to make sure that intermediate values are being computed correctly and are not being cancelled along the output path. In general, intermediate values might be wrong not because of an error in the specification or design, but because of the manufacturing process.

Observability and controllability are also introduced into the software design with the same goal: to have an observable and controllable implementation. This goal is very important and its realization implies in design and/or implementation changes. But

Work	Testing goals, factors that affect and improve testability, and achieving the testing goals
Probert & Geldrez	G: semantic coverage. I: semantic instrumentation. H: identifying equivalence classes of behavior, representing the design with design machines, establishing mapping functions, and introducing into the implementation semantic probes.
Dssouli & Fournier	G: observability and controllability. I: observable architecture of the software. H: introduction of internal interaction points during the implementation and testability phase and definition of some kind of mechanism to access the IPs.
Saleh	G: observability and controllability. A: testability requirements introduced at the protocol design. I: architectural and behavioral requirements. H: synthesis process that introduces architectural and behavioral requirements into the protocol specification.
Yu	G: definition of a formalism that can be used to write testability-directed specifications. I: a formal method that has a strong expressive power and a timing model that facilitates the testing process. H: extending TAL with relative clock time and temporal assertions.
Drira et al.	G: analyze how difficult is to test an implementation through an environment. A: environment where the implementation is embedded. H: using refusal graphs to perform the testability degradation analysis.
Petrenko et al.	G: estimate the length of a complete test suite. I: testability metric based on the length of a complete test suite. H: testability metric that considers the length of complete test suites.

Table 1: Testing goals, factors, and achieving testing goals.

note that this is just one goal. In communication protocols there are different testing goals depending on how this process is performed, the type of testing, etc. as discussed extensively by Chanson, Loureiro and Vuong in [7, 29] when they propose a framework to reason about design for testability in communication protocols.

In summary, design for testability is much more than observability and controllability and in the next section we present some guidelines that should be used when developing new DFT techniques.

7 What are the directions to follow?

It seems to us that there are four fundamental points that should be observed when developing new design for testability techniques that are general enough to apply to any

Work	DFT/T	Phases S,D,I,T	Tech. A,S	Probe effect	Metrics	Tool
Probert & Geldrez	DFT	S,D,I	A,S	N	N	N
Dssouli & Fournier	T	I	A,S	Y	N	N
Saleh	DFT	S,D	S	Y	N	N
Yu	DFT	S,I,T	S	N	N	N
Drira et al.	DFT	S,I,T	A	N	N	Y
Petrenko et al.	DFT	D,I,T	A	N	Y	N

Table 2: Other criteria to compare the methods:

problem domain. They are described in the following.

Definition of the testing goals to be accomplished. Given a specific issue that affects the testing process, testing goals should be clearly defined since they will be used to develop the testability requirements to be introduced in the design. The goals will guide the entire development process. Therefore, they should be expressed in terms of assumptions, requirements, constraints, and what is expected to be accomplished.

Identification of the factors that affect the testing goals to be accomplished. The development of a new DFT technique starts in this step. Once the factors that affect the proposed testing goals are identified and deeply understood it is possible to start reasoning about them.

Identification of the factors that can improve the testing goals to be accomplished. This is probably the most difficult part, i.e., finding the solution/factors that should be incorporated into the design. The goal is to identify these testability requirements that will lead to an implementation easier to be tested according to the testing goals.

Definition of the process to achieve the desired testing goals. This is the development process. As result it should provide either an algorithm or a methodology to implement the solution found in the previous step. Is it that simple? Definitely no. This process involves a lot of creative work and a great understanding of the entire protocol development process.

Ideally, the process of designing a new DFT technique should comprise *synthesis* and *analysis* techniques. The product of such process is the design specification that should lead to a correct and easily testable implementation. In a broader sense, synthesis denotes

the process of building an entity using primitive or predefined elements according to an algorithm. The final entity should have certain desired properties by construction. Analysis denotes the process of decomposing an entity in its elements and analyzing them using an algorithm. The goal of this process is to identify some undesirable properties or errors.

The synthesis and analysis processes should be based on a formal notation. For instance, a formal description technique, a programming language, etc. Otherwise it will be not possible to define precisely or prove how or whether the goals of synthesis and analysis can be accomplished by applying certain rules and techniques.

In the context of design for testability, synthesis denotes the process of specifying a design specification that will meet some testability requirements according to some testing goals whereas analysis denotes the process of examining a design specification to check whether some testability requirements are met according to some testing goals. The applicability of each technique will depend on the issues under consideration.

Testability is, in general, a qualitative property but depending on the testing goals may be quantitative. Whenever possible testability metrics should be given. Although, the metrics should consider both the control and data parts of the protocol. If only one part is considered the complexity of the protocol may be shifted to the other one.

It is very important to have software tools that help designers to meet testability requirements. In the hardware domain commercial tools have been available for some time now. In [8] a survey on tools used for protocol development based on formal description techniques (FDTs) is presented and none of them have facilities to incorporate testability into the design. Among the methods discussed in Section 4 only Voas and Drira et al. have tools that implement their methods. In the former case for sequential software in general and in the latter for communication protocols (prototype tool).

8 Concluding remarks

In this paper we made a critical assessment of design for testability in communication protocols. We presented five reasons that make the testing of concurrent reactive systems, i.e., communication protocols, more difficulty to be realized, described briefly how communication protocols are tested, and the different meanings of testability and design for testability. We presented a critical assessment of DFT in communication protocols and argued that design for testability is much more than observability and controllability. Finally, we presented and discussed four important points that should be considered when developing new DFT techniques.

It is hoped that this paper will facilitate and encourage further research activities in formalizing and solving DFT problems, thereby alleviating the testing process of communication protocols.

References

- [1] Richard Bache and Monika Müllerburg. Measures of testability as a basis for quality assurance. *Software Engineering Journal*, 5(2):86–92, March 1990.
- [2] Boris Beizer. *Software Testing Techniques*. Van Nostrand Reinhold, New York, NY, USA, second edition, 1990.

- [3] Robert V. Binder. Design for testability in object-oriented systems. *Communications of the ACM*, 37(9):87–101, September 1994.
- [4] Gregor von Bochmann and Alexandre Petrenko. Protocol testing: Review of methods and relevance for software testing. In Thomas Ostrand, editor, *Proceedings of the International Symposium on Software Testing and Analysis*, pages 109–124, Seattle, WA, USA, 17–19 August 1994. ACM Press. Invited Paper. Also published as Software Engineering Notes, Special issue, 1994.
- [5] B. Boehm, J. Brown, H. Kaspar, M. Lipow, J. MacLeod, and M. Merritt. *Characteristics of Software Quality*, volume 1 of *TRW Series of Software Technology*. North-Holland Publishing Company, 1978.
- [6] Barry W. Boehm. Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1):75–88, January 1984. Reprinted in Richard H. Thayer and Merlin Dorfman (eds.), *Tutorial: System and Software Requirements Engineering*, IEEE Computer Society Press, Washington, DC, USA, 1990.
- [7] S.T. Chanson, A.A.F. Loureiro, and S.T. Vuong. On the design for testability of communication software. In *Proc. of the 24th IEEE International Test Conference*, pages 190–199, Baltimore, MD, USA, 17–21 October 1993.
- [8] S.T. Chanson, A.A.F. Loureiro, and S.T. Vuong. On Tools Supporting the Use of Formal Description Techniques in Protocol Development. *Computer Networks and ISDN Systems*, 25(7):723–739, February 1993.
- [9] S.T. Chanson, A.A.F. Loureiro, and S.T. Vuong. Testing and testability in hardware and software. In *Proceedings of the Ninth International Conference on Systems Engineering*, pages 240–244, Las Vegas, Nevada, USA, July 14–16 1993.
- [10] S.T. Chanson, S.T. Vuong, and H. Dany. Ferry clip approach to multi-party testing. In *Third Int'l Workshop on Protocol Test Systems*, Amsterdam, 1990. North-Holland.
- [11] K. Drira, P. Azema, B. Soulas, and A.M. Chemali. A formal assessment of synchronous testability for communicating systems. In *Proc. of the 13th Int'l Conference on Distributed Computing Systems*, pages 149–156, Pittsburgh, PA, USA, May 1993.
- [12] K. Drira, P. Azema, B. Soulas, and A.M. Chemali. Testability of a system through an environment. In M.-C. Gaudel and J.-P. Jouannaud, editors, *Proc. of the 4th Int'l Joint Conference on the Theory and Practice of Software Development (TAPSOFT)*, Orsay, France, 13–17 April 1993. In *Lecture Notes in Computer Science* 668.
- [13] Khalil Drira, Pierre Azéma, and François Vernadat. Refusal graphs for conformance tester generation and simplification: A computational framework. In A. Danthine, G. Leduc, and P. Wolper, editors, *Protocol Specification, Testing, and Verification, XIII*, pages E2–1–E2–16, Amsterdam, 1993. North-Holland.
- [14] Rachida Dssouli and Reine Fournier. Communication software testability. In I. Davidson and W. Litwack, editors, *Third Int'l Workshop on Protocol Test Systems*, pages 45–55, Amsterdam, 1990. North-Holland.

- [15] Roy S. Freedman. Testability of software components. *IEEE Transactions on Software Engineering*, SE-17(6):553–564, June 1991.
- [16] Daniel Hoffman. Hardware testing and software ICs. In *Proc. of Pacific Northwest Software Quality Conference*, pages 234–244, 1989.
- [17] ISO IS9646: Information Processing Systems, Open Systems Interconnection. OSI Conformance Testing Methodology and Framework, 1991.
- [18] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, July 1978.
- [19] Kim G. Larsen. Ideal specification formalism = expressivity + compositionality + decidability + testability + ... In J.C.M. Baeten and J.W. Klop, editors, *CONCUR '90: Theories of concurrency-unification and extension*, pages 33–56. Volume 458 of *Lecture Notes in Computer Science*, Springer-Verlag, 1990.
- [20] D. Parnas and Y.-B. Wang. The trace assertion method of module interface specification. Technical Report 89–261, Queen's University, Ontario, Canada, 1989.
- [21] David L. Parnas, A. John van Schouwen, and Shu Po Kwan. Evaluation of safety-critical software. *Communications of the ACM*, 33(6):636–648, June 1990.
- [22] Alexandre Petrenko, Rachida Dssouli, and Hartmut Koenig. On evaluation of testability of protocol structures. In *Sixth Int'l Workshop on Protocol Test Systems*, Amsterdam, 1993. North-Holland.
- [23] R. Probert and K. Saleh. Synthesis of communication protocols: Survey and assessment. *IEEE Transactions on Computers*, C-40(4):468–476, April 1991.
- [24] Robert L. Probert and Cecilia Geldrez. Communications systems design for testability: Grey-box testing. Technical Report 90–34, Department of Computer Science, University of Ottawa, Ottawa, Canada, July 1990.
- [25] John H. Reif and Scott A. Smolka. The complexity of reachability in distributed communicating processes. *Acta Informatica*, 25(3):333–354, 1988.
- [26] Kassem Saleh. Testability-directed service definitions and their synthesis. In *Proc. of the Eleventh IEEE Phoenix Conference on Computers and Communications*, Arizona, USA, March 1992.
- [27] Richard H. Thayer and Mildred C. Thayer. Glossary. In Richard H. Thayer and Merlin Dorfman, editors, *System and Software Requirements Engineering*, pages 605–676. IEEE Computer Society Press, 1990.
- [28] Jeffrey M. Voas. Factors that affect software testability. In *Proc. of the Ninth Pacific Northwest Software Quality Conference*, pages 235–247, Portland, OR, USA, 7–8 October 1991.
- [29] S.T. Vuong, A.A.F. Loureiro, and S.T. Chanson. A framework for the design for testability of communication protocols. In *6th International Workshop on Protocol Test Systems*, pages 89–108, Pau, France, 28–30 September 1993. Published as IFIP Transactions C: Communication Systems C-19 1994, North Holland.

- [30] Werner Schütz. On the Testability of Distributed Real-Time Systems. In *Proc. of the Tenth Symposium on Reliable Distributed Systems*, pages 52–61, Pisa, Italy, 30 Sep–2 Oct 1991. Also published as Research Report 6, Technical University of Vienna, Vienna, Austria, 1991.
- [31] Lee White and Hareton K.N. Leung. Regression testability. *IEEE Micro*, 12(2):81–84, April 1992.
- [32] Hua-Long Yu. Testability-directed specification of communications software. Master's thesis, Department of Computer Science, University of Ottawa, Ottawa, Canada, 1991.